

Programozás Alapjai

Dr. Gergely Tamás
Dr. Kicsi András

Szegedi Tudományegyetem
Informatikai Intézet
Szoftverfejlesztés Tanszék

2025

(v1207)



Tartalomjegyzék

- 
- 1 Bemutatók**
 - Kurzus információk
 - Infrastruktúra szabályzatok
 - Mentorálás
 - 2 Python alapok**
 - Bevezető
 - Pénzváltás (1. verzió)
 - Pénzváltás (2. verzió)
 - 3 Alapfogalmak**
 - Alapfogalmak
 - Alapvető programelemek fogalmak
 - 4 Python feltételes ciklusok**
 - Röppálya számítás
 - Röppálya szimuláció
 - 5 Szoftverfejlesztési életciklus modellek**
 - A programozás fázisai
 - 6 Python sztringek**
 - Python sztringek
 - Basename meghatározása
 - 7 Vezérlési szerkezetek**
 - Bevezetés
 - Szekvenciális vezérlés
 - Függvények
 - 8 Python listák**
 - Python list és tuple adattípusok
 - Python listaszerű adattípusok műveletei
 - Basename és eltelt idő
 - 9 Vezérlési szerkezetek**
 - Szelekciós vezérlések
 - 10 Python iterációk**
 - Csúszóátlag számítása
 - 11 Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 12 Python halmazok, comprehension**
 - Python set és frozenset adattípusok
 - Tér legtovábbi pontjai
 - 13 Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 14 Python dictionary, függvény mint típus**
 - Python dict adattípus
 - Python comprehension
 - Python function adattípus
 - 15 Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 16 Python fájlkezelés**
 - Fájlkezelés
 - 17 Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 18 Python formátált IO**
 - Python formátált IO
 - 19 Adatkezelés, adattárolás**
 - Az adatkezelés szintjei
 - Az adattárolás jellemzői
 - 20 Python típusok**
 - Python numerikus típusok
 - 21 Absztrakt adattípusok**
 - Egyszerű adattípusok
 - Összetett adattípusok
 - 22 Python típusok**
 -
 - 23 Absztrakt adattípusok**
 - További Python típusok
 - Python function adattípus
 - 24 Tesztelés**
 - Tesztelés
 - Tesztelés Python-ban
 - A pytest
 - 25 Vizualizáció és egyéb modulok**
 - Fájlkezelő modulok
 - Vizualizáció
 - 26 More Python**
 - A Python program végrehajtásának folyamata
 - Függvényparaméterek és argumentumok
 - Python unpacking
 - Kivételkezelés

Tartalomjegyzék

- 
- 1 Bemutatók**
 - Kurzus információk
 - Infrastruktúra szabályzatok
 - Mentorálás
 - 2 Python alapok**
 - Bevezető
 - Példák (1. verzió)
 - Példák (2. verzió)
 - 3 Alapfogalmak**
 - Alapfogalmak
 - Alapvető programelemek fogalmak
 - 4 Python feltételes ciklusok**
 - Röppálya számítás
 - Röppálya szimuláció
 - 5 Szoftverfejlesztési életciklus modellek**
 - A programozás fázisai
 - 6 Python sztringek**
 - Python sztringek
 - Basename meghatározása
 - 7 Vezérlési szerkezetek**
 - Bevezetés
 - Szekvenciális vezérlés
 - Függvények
 - 8 Python listák**
 - Python list és tuple adattípusok
 - Python listák adattípusok műveletei
 - Basename és eltelt idő
 - 9 Vezérlési szerkezetek**
 - Szelekciós vezérlések
 - 10 Python iterációk**
 - Csúszólagos számítás
 - 11 Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 12 Python halmazok, comprehension**
 - Python set és frozenset adattípusok
 - Tér legtovábbi pontjai
 - 13 Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 14 Python dictionary, függvény mint típus**
 - Python dict adattípus**
 - Python comprehension
 - Python function adattípus
 - 15 Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 16 Python fájlkezelés**
 - Fájlkezelés
 - 17 Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 18 Python formátált IO**
 - Python formátált IO
 - 19 Adatkezelés, adattárolás**
 - Az adatkezelés szintjei
 - Az adattárolás jellemzői
 - 20 Python típusok**
 - Python numerikus típusok
 - 21 Absztrakt adattípusok**
 - Egyszerű adattípusok
 - Összetett adattípusok
 - 22 Python típusok**
 - 23 Absztrakt adattípusok**
 - További Python típusok
 - Python function adattípus
 - Összetett adattípusok
 - 24 Tesztelés**
 - Tesztelés
 - Tesztelés Python-ban
 - A pytest
 - 25 Vizualizáció és egyéb modulok**
 - Fájlkezelő modulok
 - Vizualizáció
 - 26 More Python**
 - A Python program végrehajtásának folyamata
 - Függvényparaméterek és argumentumok
 - Python unpacking
 - Kivételkezelés

A dict (*dictionary*) adattípus

- A *dictionary*-k asszociatív tömbök, kulcs-érték párok *listáját* tárolják.
- Felfoghatók olyan *listáknak*, amiket potenciálisan bármivel (nem csak egész számokkal) indexelhetünk.
- A dict *mutable* típus: az elemeit megváltoztathatjuk, törölhetjük, beszúrhatunk elemeket, stb.

```
>>> d = {'name': 'Garfield'}
>>> d['name']
'Garfield'
>>> d['age'] = 45
>>> d[0] = '1978'
>>> d
{'name': 'Garfield', 'age': 45, 0: '1978'}
```

A dict (*dictionary*) adattípus műveletei

Operátorok

- egyoperandusú műveletek
 - `len()` elemszám
- kétoperandusú műveletek
 - | dictionary-k „egyesítése”
 - `[]` indexelés
 - `in` kulcs megléte
 - `not in` kulcs hiánya
 - `del` kulcs szerinti törlés

```
len({'egy': 1, 2: 'kettő'})

d['három'] = 'beszúrás'
v = d[2]
'egy' in d
'kettő' not in d
del d['három']
```

A dict (*dictionary*) adattípus műveletei

Relációs műveletek

- kétoperandusú műveletek

`==` egyenlő

`!=` nem egyenlő

```
{1: "e" + "gy"} == {4-3: "egy"}  
{1: "egy"} != {"egy": 1}
```



Konverzió dict (*dictionary*) típusra

- dict(pairs)

- lista, tuple, halmaz: kulcs-érték párok összességéből készült dictionary

```
>>> dict([(1,2),(3,4),(5,6),(1,8)])
1: 2, 3: 4, 5: 6
>>> dict(((1,2),(3,4),(5,6),(1,8)))
1: 2, 3: 4, 5: 6
>>> dict({(1,2),(3,4),(5,6),(1,8)})
1: 8, 3: 4, 5: 6
```



A dict (*dictionary*) adattípus műveletei

Hasznos műveletek (módosítás)

- műveletek

- `.update()` elemek beszúrása / átírása
- `.pop()` adott kulcsú érték kivétele
- `.popitem()` egy elem kivétele (LIFO)
- `.clear()` összes kulcs törlése

```
d.update({'auto': None, 0: []})  
v = d.pop('kulcs')  
k, v = d.popitem()  
d.clear()
```



A dict (*dictionary*) adattípus műveletei

Hasznos műveletek (hozzáférés)

- hozzáférés

- `.get()` kulcshoz tartozó érték
- `.keys()` kulcsok sorozata
- `.values()` értékek sorozata
- `.items()` kulcs-érték párok sorozata
- `.copy()` másolás

```
v = d.get('bicikli')  
k = set(d.keys())  
v = list(d.values())  
for k, v in d.items(): pass  
d2 = d.copy()
```



A dict (*dictionary*) adattípus műveleteinek tulajdonságai

- A dict *mutable* típus. Bizonyos műveletek annak a dictionary-nek az értékét változtatják meg, amin dolgoznak, és nem új dictionary-t hoznak létre!

```
>>> a = b = {1: "egy", 2: "kető"}
>>> a is b
True
>>> b |= {2: "kettő", 3: "három"}
>>> a is b
True
>>> a
{1: 'egy', 2: 'kettő', 3: 'három'}
>>> b
{1: 'egy', 2: 'kettő', 3: 'három'}
```

A dict (*dictionary*) adattípus bejárása

Kulcs, érték és elemek szerint

- Egy dict értéket háromféleképpen lehet bejárni a for segítségével:
 - Kulcsok szerint, for-ral vagy a .keys() alapján
 - Értékek szerint, a .values() alapján
 - Elemek (kulcs-érték párok) szerint, az .items() alapján

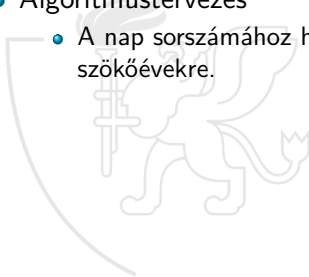
```
>>> d = {'egy':1, 'kettő':2}
>>> for k in d: print(k)
...
egy
kettő
>>> for k in d.keys(): print(k)
...
egy
kettő
```

```
>>> for v in d.values(): print(v)
...
1
2
>>> for k,v in d.items(): print(k,v)
...
egy 1
kettő 2
```

Nap sorszáma az évben

Problémafelvetés, specifikáció és algoritmustervezés

- Problémafelvetés
 - Egy adott dátum az év hányadik napjára esik?
- Specifikáció
 - Az input egy YYYY-MM-DD formában megadott dátum.
 - Az output egy szám, hogy a megadott dátum az adott év hányadik napja.
- Algoritmustervezés
 - A nap sorszámaához hozzá kell adni a megelőző hónapok napjainak számát, figyelve a szökőévekre.



Nap sorszáma az évben [1/2]

evnapja_dict.py [1-19]

```
1#!/usr/bin/env python3
2"""
3Programozás Alapjai előadás példaprogram
4SZTE TTIK, Informatikai Intézet, Szoftverfejlesztés Tanszék
5Gergely Tamás, gertom@inf.u-szeged.hu, 2024-05-31
6cc-by-nc-nd
7
8Egy YYYY-MM-DD formátumban megadott dátum az év hányadik napja?
9"""
10from datetime import datetime
11from datetime import timedelta
12
13def hanyadik_nap(yy, mm, dd):
14    napok = {1: 31, 2: 29 if leap(yy) else 28, 3: 31, 4: 30,
15             5: 31, 6: 30, 7: 31, 8: 31, 9: 30, 10: 31, 11: 30}
16    ev_napja = dd
17    for ho in range(1, mm):
18        ev_napja += napok[ho]
19    return ev_napja
```

Nap sorszáma az évben [2/2]

evnapja_dict.py [21–32]

```
21 def main():
22     datum = input("Kérem a dátumot YYYY-MM-DD alakban!")
23     (ev, ho, nap) = datum.split("-")
24     (ev, ho, nap) = (int(ev), int(ho), int(nap))
25
26     if helyes(ev, ho, nap):
27         print("%4d-%02d-%02d az év %d. napja" % (ev, ho, nap, hanyadik_nap(ev, ho, nap)))
28     else:
29         print("A %4d-%02d-%02d dátum érvénytelen!" % (ev, ho, nap))
30
31 if __name__ == "__main__":
32     main()
```



Tartalomjegyzék

- 
- 1 Bemutatók**
 - Kurzus információk
 - Infrastruktúra szabályzatok
 - Mentorálás
 - 2 Python alapok**
 - Bevezető
 - Példák (1. verzió)
 - Példák (2. verzió)
 - 3 Alapfogalmak**
 - Alapfogalmak
 - Alapvető programelemek fogalmak
 - 4 Python feltételes ciklusok**
 - Röppálya számítás
 - Röppálya szimuláció
 - 5 Szoftverfejlesztési életciklus modellek**
 - A programozás fázisai
 - 6 Python sztringek**
 - Python sztringek
 - Basename meghatározása
 - 7 Vezérlési szerkezetek**
 - Bevezetés
 - Szekvenciális vezérlés
 - Függvények
 - 8 Python listák**
 - Python list és tuple adattípusok
 - Python listák adattípusok műveletei
 - Basename és eltelt idő
 - 9 Vezérlési szerkezetek**
 - Szelekciós vezérlések
 - 10 Python iterációk**
 - Csúszólag számítása
 - 11 Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 12 Python halmazok, comprehension**
 - Python set és frozenset adattípusok
 - Tér legtovábbi pontjai
 - 13 Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 14 Python dictionary, függvény mint típus**
 - Python dict adattípus
 - Python comprehension**
 - Python function adattípus
 - 15 Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 16 Python fájlkezelés**
 - Fájlkezelés
 - 17 Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 18 Python formátált IO**
 - Python formátált IO
 - 19 Adatkezelés, adattárolás**
 - Az adatkezelés szintjei
 - Az adattárolás jellemzői
 - 20 Python típusok**
 - Python numerikus típusok
 - 21 Absztrakt adattípusok**
 - Egyszerű adattípusok
 - Összetett adattípusok
 - 22 Python típusok**
 - További Python típusok
 - Python function adattípus
 - 23 Absztrakt adattípusok**
 - Összetett adattípusok
 - 24 Tesztelés**
 - Tesztelés
 - Tesztelés Python-ban
 - A pytest
 - 25 Vizualizáció és egyéb modulok**
 - Fájlkezelő modulok
 - Vizualizáció
 - 26 More Python**
 - A Python program végrehajtásának folyamata
 - Függvényparaméterek és argumentumok
 - Python unpacking
 - Kivételkezelés

Comprehension

list, set és dict típusok esetében

- A Python nyelv egy erős és tömör eszköze a *comprehension*, amivel matematikai jelölésmóddal és logikával tudunk *iterálható* típusú értékekből származtatott list, set, vagy dict értékeket készíteni.
 - Az n^n -ként kifejezhető számok halmaza, ahol $n \in \mathbb{N}$:
 $\{n^n : n \in \mathbb{N}\} \rightarrow \{ n ** n \text{ for } n \text{ in } \mathbb{N} \}$
 - Az $[0 \dots n)$ intervallumba eső egész számokból képezhető kéttagú szorzatok halmaza:
 $\{ab : a, b \in [0, n)\} \rightarrow \{ a * b \text{ for } a \text{ in range}(n) \text{ for } b \text{ in range}(n) \}$
 - Az $\mathbb{R}$ halmazban lévő negatív számok halmaza:
 $\{x \in \mathbb{R} : x < 0\} \rightarrow \{ x \text{ for } x \text{ in } \mathbb{R} \text{ if } x < 0 \}$
 - Egy L sorozat páros elemeit követő elemei indexeinek listája:
 $\{i : i \in \mathbb{N}_+, L_{i-1} \text{ páros}\} \rightsquigarrow [i+1 \text{ for } i \text{ in range}(\text{len}(L)-1) \text{ if } L[i] \% 2 == 0]$
 - $x \mapsto f(x)$ hozzárendelések halmaza, ahol x egy V halmaz eleme, az $(x, f(x))$ pont pedig az I. vagy III. síknegyedben van:
 $\{x \mapsto f(x) : x \in V, x < 0 \wedge f(x) < 0 \vee 0 < x \wedge 0 < f(x)\} \rightsquigarrow$
 $\{ x:f(x) \text{ for } x \text{ in } V \text{ if } x < 0 \text{ and } f(x) < 0 \text{ or } 0 < x \text{ and } 0 < f(x) \}$

Comprehension

list, set és dict típusok esetében

- Példák

```
>>> Z = [-1, 1, 3, 2, 0, -2, -3]
>>> N = [n - min(Z) for n in Z]
>>> N
[2, 4, 6, 5, 3, 1, 0]
>>> P = [n for n in Z if n % 2 == 0]
>>> P
[2, 0, -2]
>>> S = {n: n ** 2 for n in Z}
>>> S
{-1: 1, 1: 1, 3: 9, 2: 4, 0: 0, -2: 4, -3: 9}
>>> X = {s for s in S.values()}
>>> X
{0, 1, 4, 9}
```

Tartalomjegyzék

- 
- 1 Bemutatók**
 - Kurzus információk
 - Infrastruktúra szabályzatok
 - Mentorálás
 - 2 Python alapok**
 - Bevezető
 - Pénzváltás (1. verzió)
 - Pénzváltás (2. verzió)
 - 3 Alapfogalmak**
 - Alapfogalmak
 - Alapvető programelemek fogalmak
 - 4 Python feltételes ciklusok**
 - Röppálya számítás
 - Röppálya szimuláció
 - 5 Szoftverfejlesztési életciklus modellek**
 - A programozás fázisai
 - 6 Python sztringek**
 - Python sztringek
 - Basename meghatározása
 - 7 Vezérlési szerkezetek**
 - Bevezetés
 - Szekvenciális vezérlés
 - Függvények
 - 8 Python listák**
 - Python list és tuple adattípusok
 - Python listáshoz adattípusok műveletei
 - Basename és eltelt idő
 - 9 Vezérlési szerkezetek**
 - Szelekciós vezérlések
 - 10 Python iterációk**
 - Csúszólag számítása
 - 11 Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 12 Python halmazok, comprehension**
 - Python set és frozenset adattípusok
 - Tér legtávolabbi pontjai
 - 13 Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 14 Python dictionary, függvény mint típus**
 - Python dict adattípus
 - Python comprehension
 - Python function adattípus
 - 15 Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 16 Python fájlkezelés**
 - Fájlkezelés
 - 17 Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 18 Python formátált IO**
 - Python formátált IO
 - 19 Adatkezelés, adattárolás**
 - Az adatkezelés szintjei
 - Az adattárolás jellemzői
 - 20 Python típusok**
 - Python numerikus típusok
 - 21 Absztrakt adattípusok**
 - Egyszerű adattípusok
 - Összetett adattípusok
 - 22 Python típusok**
 - További Python típusok
 - Python function adattípus
 - 23 Absztrakt adattípusok**
 - Összetett adattípusok
 - 24 Tesztelés**
 - Tesztelés
 - Tesztelés Python-ban
 - A pytest
 - 25 Vizualizáció és egyéb modulok**
 - Fájlkezelő modulok
 - Vizualizáció
 - 26 More Python**
 - A Python program végrehajtásának folyamata
 - Függvényparaméterek és argumentumok
 - Python unpacking
 - Kivételkezelés

A function (függvény) adattípus

- A függvényre eddig mint algoritmusra gondoltunk.
- A Neumann-architektúra szerint az algoritmusok a számítógépben ugyanúgy (bitsorozatként) és ugyanott (memóriában) tárolódnak mint az adatok.
- A függvény azonosító (név) tulajdonképpen az algoritmus implementációját azonosítja.
- Miért ne adhatnánk ezt az algoritmust értékül egy másik azonosítónak (változónak)?

```
>>> def f(x):  
...     return x ** x  
...  
>>> variable = f  
>>> variable(3)  
27  
>>> variable(4)  
256
```

A function (függvény) adattípus műveletei

Operátorok, reláció

- műveletek
 - () függvényhívás
- összehasonlítás
 - `==` ugyanaz a függvény
 - `!=` másik függvény

```
fgv(x, y)
```

```
fgv == f
```

```
fgv != min
```



Rendezés több szempont szerint

Problémafelvetés és specifikáció

- Problémafelvetés
 - Adott hallgatók egy listája a számonkéréseken elért átlagpontszámaikkal.
 - Rendezzük a listát több szempont szerint is és minden rendezés után legyen kiíratás!
- Specifikáció
 - Az input név-adat párok sorozata, amelynek végét egy üres név jelzi.
 - Az output a név szerint növekvő, adat szerint növekvő illetve csökkenő sorrendben rendezett három lista.



Rendezés több szempont szerint

Algoritmustervezés

- Az első megvalósításnál kihasználjuk a Python `list` típusának rendező metódusát, amely rendezési szempontként egy olyan függvényt vár, ami a lista egy elemére egy relációs művelettel rendelkező típusú értéket ad vissza.



Rendezés [1/2]

rendezes.py [1–26]

```
1#!/usr/bin/env python3
2"""
3Programozás_Alapjai_előadás_példaprogram
4SZTE_TTIK,_Informatikai_Intézet,_Szoftverfejlesztés_Tanszék
5Gergely_Tamás,_gertom@inf.u-szeged.hu,_2024-05-31
6cc-by-nc-nd
7
8Rendezzük_névsorba_illetve_átlag_szerint_a_hallgatókat!
9"""
10def beolvas():
11    n = input("Név? ")
12    if not n:
13        return None
14    p = float(input("Átlag? "))
15    return {'nev': n, 'adat': p}
16
17def kiiras(msg, l):
18    print(msg)
19    for e in l:
20        print("%s: %.2f" % (e['nev'], e['adat']))
21
22def nev(x):
23    return x['nev']
24
25def adat(x):
26    return x['adat']
```

Rendezés [2/2]

rendezes.py [28–45]

```
28 def main():
29     sor = []
30     while (e := beolvas()) is not None:
31         sor.append(e)
32         print(e)
33     kiiras("Eredeti_sorrend:", sor)
34
35     sor.sort(key=nev)
36     kiiras("Névsor_szerint_rendezve:", sor)
37
38     sor.sort(key=adat)
39     kiiras("Adat_szerint_rendezve:", sor)
40
41     sor.sort(key=adat, reverse=True)
42     kiiras("Adat_szerint_csökkenő_sorba_rendezve:", sor)
43
44 if __name__ == "__main__":
45     main()
```